



Caruca v2

LLM-based specification mining for opaque shell commands

Experiment Designs

Version 1.1 — corrected 22 September 2026

SOURCE `ai_docs/analysis/experiment_designs.md`

GENERATED September 22, 2026 at 4:34 PM

PAGES 11

CARUCA V2 — RESEARCH PROJECT

Table of contents

Experiment Designs — Turning the Scientific Candidates Into Runnable Work

E0. Pin the artifact set — a reproducibility check that has to happen first

E1. How far do hand-written specifications hold, and does the gap matter?

E2. Does the model read the documentation, or remember it?

E3. What do the extrapolation tiers actually assume?

E4. Which fixture axes actually matter?

E5. A ground-truth generator

E6. Downstream utility, and downstream harm

Suggested order

What is deliberately not here

Experiment Designs — Turning the Scientific Candidates Into Runnable Work

Version 1.1 — note added 22 September 2026. Two clarifications, no design changed. (1) E0's three motivating bullets below were **superseded** by `e0_artifact_pinning.md`, which retired the diverged-branch worry and resolved the `grep` provenance; read that document's findings, not these bullets, as the current state. (2) Where these designs call output-hash comparison "the same method the paper already uses", that refers to the paper's *unnumbered* PaSh re-run check — **not** to the method behind its 52/52 figure, which is a per-command hand comparison against PaSh's hand-written annotations. And because the paper reports no performance figure anywhere (the word "speedup" does not occur in it), E6's speedup results are a wholly new measurement with no published baseline, not a reproduction.

Protocol-level designs for the contributions identified in `scientific_vs_engineering_contributions.md`. That document argues *what would count as science*; this one says *what to actually run*, what already exists to run it with, and what result would falsify each hypothesis.

Ordered by dependency, not by importance. **E0 is a prerequisite for E1 and is not optional** — see below.

Every claim about existing infrastructure was checked against v1 at commit `d80324073` on branch `main` (`~/dev/stevens/caruca` on this Mac, `~/stevens/caruca` on the WSL PC). Nothing here has been executed; effort estimates are for a single person and assume the Lima tracing VM is working (see `mac_lima_tracing_env`).

E0. Pin the artifact set — a reproducibility check that has to happen first

Why this exists. While grounding E1, three things on `main` did not line up with what the paper reports. None of these is proof of an error; all three are reasons to pin exact revisions before building an experiment on top of them.

1. **The paper says `grep` “is marked as always stateless in the original specifications.”**

The hand-written PaSh annotation in the repo,

`benchmarks/annotations/pash/grep.json`, has four cases, and the *first* one is `-c → pure`. That file does handle `-c`. Its README describes the folder as “examples of annotations our system should be able to generate,” sourced from `github.com/binpash/annotations` — so it may be a corrected or newer sample rather than the annotation set the paper measured against.

2. **The paper says `cp` is “marked as side-effectful, but in reality being pure.”** The generated annotation at `eval/pash-annotations/cp.json` marks all four of its cases `side-effectful` — i.e. the generated artifact on `main` agrees with the annotation the paper flags as divergent.

3. **The paper’s experiments were run on a different branch.** `caruca-experiments` exists on the remote alongside `main`, `detailed_annotations`, `extracting-anno`, and others. The artifacts on `main` are not necessarily the ones the paper’s numbers came from — `caruca/save/*.json` holds noticeably richer output (compound and predicates, real pipe file descriptors) than `eval/pash-annotations/*.json` does.

What to do. For each of the paper’s specific claims, record: the exact Caruca commit and branch that produced the generated specification, the exact upstream revision of the hand-written annotation it was compared against, and the command line that produced the comparison. Then re-run and confirm the three named discrepancies (`grep -c`, `ps`, `cp`) still reproduce.

Effort: 1–2 days. **Blocking for:** E1.

Why it is worth doing even if everything reproduces: “we can reproduce our own headline discrepancies from a pinned artifact” is the first thing an artifact-evaluation committee checks, and the first thing a skeptical reviewer tries. If the annotations were updated upstream *after* Caruca surfaced the divergences, that is not a problem — **it is a stronger result**, because it means the tool’s findings were already adopted. But it has to be documented as such rather than left for a reviewer to discover.

E1. How far do hand-written specifications hold, and does the gap matter?

Implements S1.

Hypothesis: hand-written command specifications shipped by PaSh, POSH, Shellcheck, and Shseer diverge from observed command behavior at a measurable rate, and a subset of those divergences are soundness-critical — they can change what a parallelized script computes.

Null result, and why it is still publishable: if the divergence rate is near zero, the crowd-sourcing proposal those papers make is validated, and the argument for automation shifts cleanly from *correctness* to *cost*. Either outcome is reportable. Do not run this experiment unless you are willing to publish the null.

Procedure:

1. Take the *complete* shipped annotation set for each consumer system — not the subset inside each system’s own benchmark suite. The paper deliberately scoped to that subset, which is the right choice for the question it was asking — so this is new ground rather than a correction.
2. Run Caruca over every annotated command and diff generated against hand-written, per invocation rather than per command.
3. Classify each disagreement: over-general flag handling, missing flag-conditional behavior, imprecise parallelizability class, stale relative to the installed command version, or *Caruca is wrong* (this bucket must exist and be reported).
4. For each disagreement where observed behavior is the more accurate account, build a **witness**: a runnable script where PaSh using the hand-written annotation and PaSh using observed behavior produce different output. Verify by hash comparison, the same method the paper already uses.

Already exists: the hand-written sets at `benchmarks/annotations/` (`pash/`, `posh.txt`, `shseer/`) and `eval/shellcheck/checks.csv`; generated output at `caruca/save/` and `eval/pash-annotations/`; PaSh’s benchmark suites at `eval/pash-scripts/` (177 shell scripts across 17 suites) for witness construction; `eval/cmp_specs.py` for pairwise comparison, though note it compares *syntax* specs, not parallelizability annotations, and `caruca_v1_eval_tooling_notes` records a denominator bug in it.

Needs building: a differential harness over annotation semantics rather than syntax; the error taxonomy; the witness generator. The witness generator is the same machinery as gap 2 in `evaluation_gaps.md` — running split invocations concurrently — which is why that gap stops being “one more test mode” here.

Effort: 2–3 weeks after E0. **Produces:** the motivating finding for Paper A.

Handle with care: the authors of those systems are prospective co-authors. Report the *Caruca is wrong* bucket with the same prominence as the others.

E2. Does the model read the documentation, or remember it?

Implements S2. Independent of E0 and E1 — can run in parallel.

Hypothesis: measured syntax-inference accuracy on GNU coreutils overstates the model’s ability to read unfamiliar documentation, because those man pages are in the training data.

Falsification: accuracy holds under renaming and under semantic permutation. That would be a genuinely surprising and valuable result — it would mean the pipeline generalizes to undocumented in-house tooling, which is the strongest possible argument for the whole approach.

Procedure — four arms, run against the same scoring harness:

ARM	CONSTRUCTION	WHAT IT ISOLATES
A. Baseline	Unmodified coreutils man pages	Reproduces the paper’s number; sanity check
B. Renamed	Consistently rewrite command and flag names — <code>cat</code> → <code>zorp</code> , <code>-n</code> → <code>--enumerate</code> — preserving described semantics	Recall of <i>names</i> . Drop here = surface memorization
C. Permuted	Keep real names, swap what flags do (make <code>-n</code> behave as <code>-s</code> described)	Recall of <i>semantics</i> . Snapping back to the real spec is the signal
D. Synthetic	Mechanically generated CLI tools, man pages generated from the implementation	Uncontaminated ceiling; ground truth correct by construction

Report accuracy per arm with confidence intervals over repeated sampling, not single runs — the paper’s 99.7% and 116-of-120 are both single-run numbers.

Already exists: documentation sources at `caruca/src/caruca/doc_sources/` (`man/` , `helpfiles/` , `versions/`) and hand-annotated ground truth at `doc_sources/ground-truth/` — the output of the 80 person-hours, in a simple JSON schema (`cat.json` is 12 flags with `short / long` arrays). The scoring harness is `eval/cmp_specs.py` plus `eval/syntax-spec-correctness.sh`.

Needs building: the mutation harness for arms B and C — a rename map applied consistently across a man page, and a semantics permuter. Arm D needs the generator described in E5.

Effort: arms A-C, 1-2 weeks. Arm D depends on E5.

Note on scope: this is core v2 work, not an add-on — model and decoding-parameter selection is already established as in scope (see `feedback_nlp_config_scope`), and v1 has no configuration surface at all to vary.

E3. What do the extrapolation tiers actually assume?

Implements S3 — the technical core of Paper A.

The structure already exists in code. `caruca/src/caruca/ir/syntax.py` defines the hierarchy: `Exact` versus `Extrapolated`, the latter splitting into `AliasStrategy`, `ShortFlagSplat`, `DefaultArg`, `ArityStrategy`, and `Agnosticism` (itself `PathAgnosticism`, `IntAgnosticism`, `StringAgnosticism`). `caruca/src/caruca/oracle.py` records which strategies fired per invocation and exposes `--exclude` over them. `eval/user-scripts/completeness_accumulate.sh:10-14` then builds the paper's coverage figure by removing filters one tier at a time — `no_assumption`, `flag_match`, `path_match`, `int_match`, `string_match` — with `grep -v`.

That is a lattice of increasingly strong assumptions, implemented as text filtering, reported as tiers (10% exact → 18% with path and integer → 94% with string → 97.78% with arity), and never given a meaning.

Two halves, and the second is the experiment:

(a) Formal. State what each extrapolation assumes, as a property of the command under test. `StringAgnosticism` assumes argument *content* does not affect the property being inferred. `ArityStrategy` assumes behavior is uniform in the number of repeated arguments.

`PathAgnosticism` assumes only the path's *kind* matters, not its name. Define a specification's validity domain compositionally from the steps used to derive it, and state downstream soundness as a theorem conditional on those assumptions.

(b) Empirical — this is the falsifiable part. Each assumption is a *claim about commands*, so test it:

- For what fraction of commands does argument content change the inferred property? Take commands where `StringAgnosticism` fired, vary content across the fixture axes, and measure how often the derived class or filesystem effect changes.
- Same for arity: does behavior actually stay uniform as repeated-argument count grows?
- Same for path naming.

Output: the 97.78% figure becomes typed rather than blended — which is the presentational fix already flagged as gap 13 in `evaluation_gaps.md`, now with a measurement behind it. And PaSh becomes configurable by soundness policy: *use only specifications at tier k or stronger*, with a measured cost in missed optimizations. That is a real soundness-versus-coverage tradeoff curve, and it is the deliverable a PL reviewer will care about most.

Already exists: the strategy instrumentation, the tier scripts, `eval/user-scripts/oracle_results.txt` holding the raw per-invocation strategy log, and the 659-line real-world invocation corpus at `eval/command-invocations.txt`.

Needs building: the formal statement (paper work, not code), and a per-assumption sensitivity sweep. That sweep is the same machinery as the fixture-axis ablation in E4 — build it once.

Effort: 3–4 weeks including the write-up. **This is the highest-value item for the resubmission.**

E4. Which fixture axes actually matter?

Converts gaps 1, 5, 6, 7, 8, 10, 12, and 14 from a to-do list into one empirical result.

Hypothesis: the fixture dimensions v1 does not vary — symlinks, permission bits, empty content, integer magnitude, environment variables, directory nesting, correlated multi-file inputs, structured content types — differ sharply in how often they change an inferred property, and the distribution is skewed enough to direct where fixture budget should go.

Why this framing and not “we added eight fixture types”: adding a fixture axis is engineering. *Measuring which axes change the answer, and by how much*, is an empirical model — the result type with the highest acceptance ratio in Shaw’s ICSE data. It also produces guidance other people can use without adopting Caruca.

Procedure: implement each axis behind the existing `--content` style knob, then ablate: for each axis, hold everything else fixed and measure the fraction of commands whose derived specification changes when that axis is varied. Report per-axis, and report the interaction terms that matter (permissions × recursive flags, for instance).

Expected shape of the result: something like “permission state changes observed behavior for X% of coreutils; content type for only Y%” — with the surprise being *which* axes turn out to be cheap and which are load-bearing.

Already exists: the environment model at `caruca/src/caruca/ir/environment.py` and content fixtures at `caruca/src/caruca/ir/contents.py` — five fixed blobs today (text, math, time, JSON, JPEG), all non-empty.

Effort: 3–4 weeks, and it shares the sensitivity harness with E3(b).

E5. A ground-truth generator

Implements S9. Enabling condition for E2 arm D.

The bottleneck: two graduate students spent 80 person-hours annotating 120 commands and had to reconcile 16 disagreements between themselves. Every future evaluation inherits both the cost and the residual uncertainty.

The build: generate CLI programs with randomized but realistic flag structures — mixed short and long forms, aliases, optional and repeated arguments, typed values — then generate their man pages *from* the implementation. Ground truth is exact by construction, quantity is unlimited, and contamination is impossible because the tools did not exist before you made them.

Validation that the generator is realistic: compare its flag-structure distribution against the real corpus at `eval/command-invocations.txt` and the flag counts at `eval/user-scripts/flag_counts/`. A generator that produces unrealistically simple interfaces would inflate arm D and has to be ruled out.

Effort: 2–3 weeks. **Also a standalone community artifact.**

E6. Downstream utility, and downstream harm

Implements S7. The validation section of Paper A.

Two numbers, and the second matters more:

- *N*: real-world scripts that PaSh or Shseer can analyze with mined specifications but could not before, with measured speedups and bugs found.
- *E*: cases where a mined specification causes a **divergent result** — different output from the unoptimized script.

E is the number that decides whether any of this is deployable, and nobody has reported it. A small nonzero *E* with a diagnosis is a stronger paper than a suspicious zero.

Corpus: `eval/pash-scripts/` already holds 177 scripts across 17 suites (oneliners, unix50, web-index, analytics-mts, bio, nlp, smoosh, and others). Extend beyond standalone shell scripts to GitHub Actions `run:` blocks and Dockerfile `RUN` lines — gap 4 in [evaluation_gaps.md](#) — and report per-population, since flag distributions differ between them.

Effort: 2–3 weeks, mostly harness and runtime.

Suggested order

PHASE	RUN	RATIONALE
1	E0	Blocking. Cheap. Prevents building on unpinned artifacts.
1 (parallel)	E2 arms A–C	Independent, fast, and publishable alone as Paper B.
2	E3	The technical core. Shares its sensitivity harness with E4.
2 (parallel)	E5	Unblocks E2 arm D; standalone artifact.
3	E1	Needs E0; needs the witness generator.
3	E4	Shares harness with E3.
4	E6	Validation; wants E1 and E3 settled first.

Paper B (E2 + E5, optionally E6's documentation-divergence angle) can be submitted while Paper A's pieces are still running.

What is deliberately not here

No experiment measures lines of code, wall-clock speedup, or token cost as a headline. Those are recorded as instrumentation — see [data_telemetry_schema](#) — and appear in the papers only where they are load-bearing: query complexity in E3's tradeoff curve, and a single automation-cost-versus-80-person-hours comparison that the original paper gestures at but never makes. The reasoning is in Part 4 of [scientific_vs_engineering_contributions.md](#).